

# MECH5200 Numerical Methods for PDEs : *Video 23:* *One Dimensional FEM : Implementation MATLAB*

David Willis

April 1, 2026

# References and Acknowledgements

The following materials were used in the preparation of this lecture:

- ① 16.920, Lecture Notes
- ② Strang & Fix: An analysis of the Finite Element Method
- ③ Bathe, Hughes, etc. have introductory books that are useful.
- ④ : Additional link to MIT 19.901, Undergraduate Numerical Methods Course – may be useful.

<http://ocw.mit.edu/courses/aeronautics-and-astronautics/16-901>

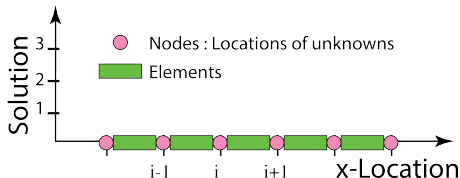
The author of these slides wishes to thank these sources for making the current lecture.

# Recall: 1-D Linear Hat Basis function

- Let's look at the steps we took for finite differences – poisson's equation:
  - 1 Discretize the domain
  - 2 Discretize the equation
  - 3 Write an equation/plan for (either):
    - 1 Nodal Assembly: each node of the domain
    - 2 Element Assembly: each element in the domain
  - 4 Construct an 'A-matrix'
  - 5 Construct a RHS matrix
  - 6 Set boundary conditions
  - 7 Solve, post-process

# STEP 1: Discretize the domain

- We begin as usual, and discretize the domain into  $N$ -intervals and  $N + 1$  nodes:



- Note, let's use equally spaced intervals for this implementation.
- in MATLAB:
  - Define how many nodes,  $N$ , there will be.
  - Define the location,  $x$  of the nodes.
  - Define how nodes are connected (optional in 1-D).

## STEP 2: Discretize the equation (here: Poisson's Equation)

- The Poisson equation in one dimension:

$$\frac{\partial^2 u}{\partial x^2} = f \quad (1)$$

- Recall, let's re-arrange the equation:

$$\frac{\partial^2 u}{\partial x^2} - f = R = 0 \quad (2)$$

- The equation has a *zero residual*,  $R$  when solved exactly.
- We really want our numerical method to *minimize* any error/residual.

# Discretizing the weak form of the equation

- See prior video lectures: The weak form of the PDE is ( $w_i$  = weighting/test function):

$$\underbrace{w_i \frac{\partial u}{\partial x} \Big|_{x_L}^{x_R}}_{\text{at boundaries}} - \underbrace{\int_{x_L}^{x_R} \frac{\partial w_i}{\partial x} \frac{\partial u}{\partial x} dx}_{a(u, w_i)} - \underbrace{\int_{x_L}^{x_R} w_i f dx}_{f(w_i)} = 0 = \underbrace{\int_{x_L}^{x_R} w_i R dx}_{\text{Weighted-Residual}} \quad (3)$$

- We can introduce piecewise continuous linear-hat basis functions for approximating the solution and also for the weighting/test functions:

$$\hat{u}(x) = \sum_{j=1}^{j=NV} u_j \cdot \phi_j \quad \text{and} \quad w_i = \phi_i \quad (4)$$

- The weighted residual equals zero – this indicates that any residual (error between the numerical solution and true solution) will be orthogonal to the weighting/test/solution basis function space.

# Discretizing the weak form of the equation

- This leads us to the following discrete expression corresponding to the weighting/test basis  $\phi_i$ :

$$\underbrace{\phi_i \frac{\partial u}{\partial x} \Big|_{x_L}^{x_R}}_{BCs: [A][u]} - \underbrace{\sum_{j=1}^{NV} u_j \int_{x_L}^{x_R} \left( \frac{\partial \phi_i}{\partial x} \cdot \frac{\partial \phi_j}{\partial x} \right) dx}_{\text{row}_i \text{ of } [A][u] : \text{internal nodes}} = \underbrace{\int_{x_L}^{x_R} (\phi_i f) dx}_{RHS_i}$$

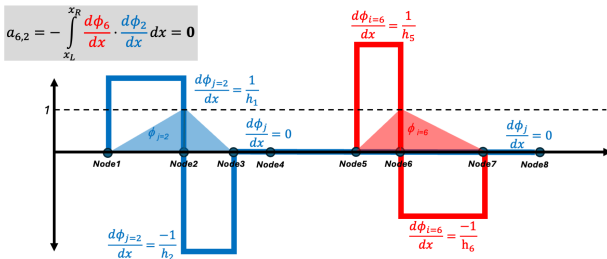
- As complicated as this looks, it isn't so bad. Let's look at 2 different ways to approach the **assembly** of the A-matrix (for the internal nodes):
  - 1 **Node-by-node:** Examine node-by-node how the weighting function  $\phi_i$  interacts with the solution basis functions  $\phi_j$ .
  - 2 **Elementwise:** Examine how the relevant nodal weighting functions  $\phi_i$  and  $\phi_{i+1}$  interact with the solution basis functions  $\phi_j$  and  $\phi_{j+1}$

# A-Matrix Assembly: Idea Node-by-node assembly

- Let's focus on this integral for the weighting basis/solution basis interaction:

$$\sum_{j=1}^{NV} u_j \underbrace{\int_{x_L}^{x_R} \left( \frac{\partial \phi_i}{\partial x} \cdot \frac{\partial \phi_j}{\partial x} \right) dx}_{a_{i,j}} = \underbrace{\begin{bmatrix} : & : & : & \dots & : \\ a_{i,1} & a_{i,2} & a_{i,3} & \dots & a_{i,n} \\ : & : & : & \ddots & : \\ : & : & : & \dots & : \end{bmatrix}}_{\text{row}_i \text{ of } [A]} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix}$$

- For test function  $\phi_i = \phi_6$  we will cycle through all  $\phi_j$  solution basis functions:



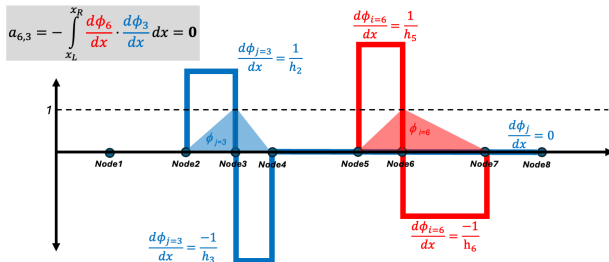


# A-Matrix Assembly: Idea Node-by-node assembly

- Let's focus on this integral for the weighting basis/solution basis interaction:

$$\sum_{j=1}^{NV} u_j \underbrace{\int_{x_L}^{x_R} \left( \frac{\partial \phi_i}{\partial x} \cdot \frac{\partial \phi_j}{\partial x} \right) dx}_{a_{i,j}} = \underbrace{\begin{bmatrix} : & : & : & \dots & : \\ a_{i,1} & a_{i,2} & a_{i,3} & \dots & a_{i,n} \\ : & : & : & \ddots & : \\ : & : & : & \dots & : \end{bmatrix}}_{\text{row}_i \text{ of } [A]} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix}$$

- For test function  $\phi_i = \phi_6$  we will cycle through all  $\phi_j$  solution basis functions:

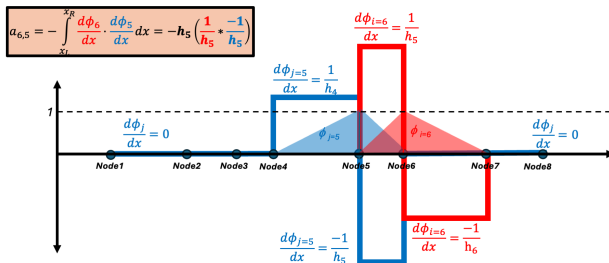


# A-Matrix Assembly: Idea Node-by-node assembly

- Let's focus on this integral for the weighting basis/solution basis interaction:

$$\sum_{j=1}^{NV} u_j \underbrace{\int_{x_L}^{x_R} \left( \frac{\partial \phi_i}{\partial x} \cdot \frac{\partial \phi_j}{\partial x} \right) dx}_{a_{i,j}} = \underbrace{\begin{bmatrix} : & : & : & \dots & : \\ a_{i,1} & a_{i,2} & a_{i,3} & \dots & a_{i,n} \\ : & : & : & \ddots & : \\ : & : & : & \dots & : \end{bmatrix}}_{\text{row}_i \text{ of } [A]} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix}$$

- For test function  $\phi_i = \phi_6$  we will cycle through all  $\phi_j$  solution basis functions:

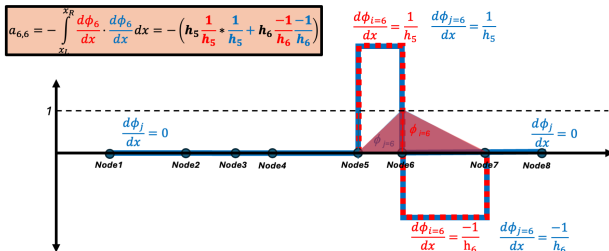


# A-Matrix Assembly: Idea Node-by-node assembly

- Let's focus on this integral for the weighting basis/solution basis interaction:

$$\sum_{j=1}^{NV} u_j \underbrace{\int_{x_L}^{x_R} \left( \frac{\partial \phi_i}{\partial x} \cdot \frac{\partial \phi_j}{\partial x} \right) dx}_{a_{i,j}} = \underbrace{\begin{bmatrix} : & : & : & \dots & : \\ a_{i,1} & a_{i,2} & a_{i,3} & \dots & a_{i,n} \\ : & : & : & \ddots & : \\ : & : & : & \dots & : \end{bmatrix}}_{\text{row}_i \text{ of } [A]} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix}$$

- For test function  $\phi_i = \phi_6$  we will cycle through all  $\phi_j$  solution basis functions:

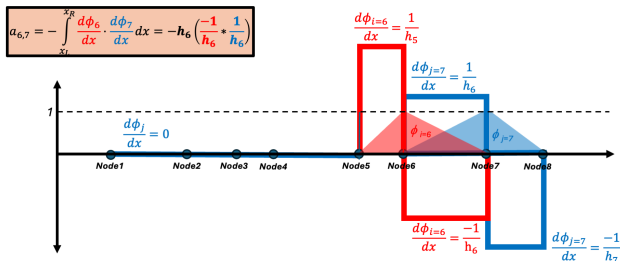


# A-Matrix Assembly: Idea Node-by-node assembly

- Let's focus on this integral for the weighting basis/solution basis interaction:

$$\sum_{j=1}^{NV} u_j \underbrace{\int_{x_L}^{x_R} \left( \frac{\partial \phi_i}{\partial x} \cdot \frac{\partial \phi_j}{\partial x} \right) dx}_{a_{i,j}} = \underbrace{\begin{bmatrix} : & : & : & \dots & : \\ a_{i,1} & a_{i,2} & a_{i,3} & \dots & a_{i,n} \\ : & : & : & \ddots & : \\ : & : & : & \dots & : \end{bmatrix}}_{\text{row}_i \text{ of } [A]} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix}$$

- For test function  $\phi_i = \phi_6$  we will cycle through all  $\phi_j$  solution basis functions:



# A-Matrix Assembly: Idea Node-by-node assembly

- Let's focus on this integral for the weighting basis/solution basis interaction:

$$\sum_{j=1}^{NV} u_j \underbrace{\int_{x_L}^{x_R} \left( \frac{\partial \phi_i}{\partial x} \cdot \frac{\partial \phi_j}{\partial x} \right) dx}_{a_{i,j}}$$

- Which, when assembled, becomes the following A-matrix:

$$\begin{bmatrix} \frac{-1}{h_1} & \frac{1}{h_1} & 0 & 0 & 0 & 0 & \dots & 0 \\ \frac{1}{h_1} & \left( \frac{-1}{h_1} + \frac{-1}{h_2} \right) & \frac{1}{h_2} & 0 & 0 & 0 & \dots & 0 \\ 0 & \frac{1}{h_2} & \left( \frac{-1}{h_2} + \frac{-1}{h_3} \right) & \frac{1}{h_3} & 0 & 0 & \dots & 0 \\ 0 & 0 & \frac{1}{h_3} & \left( \frac{-1}{h_3} + \frac{-1}{h_4} \right) & \frac{1}{h_4} & 0 & \dots & 0 \\ 0 & 0 & 0 & \frac{1}{h_4} & \left( \frac{-1}{h_4} + \frac{-1}{h_5} \right) & \frac{1}{h_5} & \dots & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{h_5} & \left( \frac{-1}{h_5} + \frac{-1}{h_6} \right) & \frac{1}{h_6} & \dots \\ : & : & : & : & : & : & : & \dots \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \\ u_n \end{bmatrix}$$

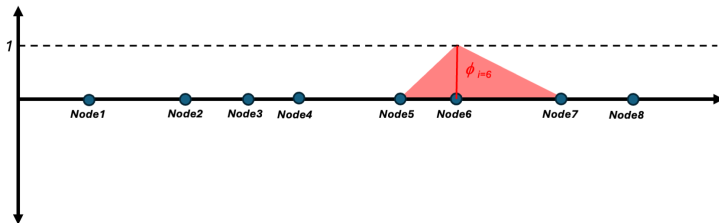
# Let's work on the RHS

- Consider the integral that will become the RHS vector:

$$RHS_i = \int_{x_L}^{x_R} (\phi_i f) dx$$

- This is the area under the basis function:

$$RHS_6 = \int_{x_L}^{x_R} \phi_6 \cdot f dx = \frac{h_5}{2} f_6 + \frac{h_6}{2} f_6$$



# Let's work on the RHS

- The result is the following RHS vector:

$$RHS_i = \int_{x_L}^{x_R} (\phi_i f) dx = \begin{bmatrix} \frac{f \cdot h_1}{2} \\ \frac{f \cdot h_1}{2} + \frac{f \cdot h_2}{2} \\ \frac{f \cdot h_2}{2} + \frac{f \cdot h_3}{2} \\ \frac{f \cdot h_3}{2} + \frac{f \cdot h_4}{2} \\ \frac{f \cdot h_4}{2} + \frac{f \cdot h_5}{2} \\ \frac{f \cdot h_5}{2} + \frac{f \cdot h_6}{2} \\ \frac{f \cdot h_6}{2} + \frac{f \cdot h_7}{2} \\ \vdots \\ \frac{f \cdot h_N}{2} \end{bmatrix}$$

# A-Matrix Assembly: Summary

- Looking at the matrix and RHS, we notice a pattern:

$$\begin{bmatrix}
 \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
 \frac{1}{h_1} & \left(-\frac{1}{h_1} + \frac{-1}{h_2}\right) & \frac{1}{h_2} & 0 & 0 & 0 & \dots & 0 \\
 0 & \frac{1}{h_2} & \left(-\frac{1}{h_2} + \frac{-1}{h_3}\right) & \frac{1}{h_3} & 0 & 0 & \dots & 0 \\
 0 & 0 & \frac{1}{h_3} & \left(-\frac{1}{h_3} + \frac{-1}{h_4}\right) & \frac{1}{h_4} & 0 & \dots & 0 \\
 0 & 0 & 0 & \frac{1}{h_4} & \left(-\frac{1}{h_4} + \frac{-1}{h_5}\right) & \frac{1}{h_5} & \dots & 0 \\
 0 & 0 & 0 & 0 & \frac{1}{h_5} & \left(-\frac{1}{h_5} + \frac{-1}{h_6}\right) & \frac{1}{h_6} & \dots \\
 \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots
 \end{bmatrix}
 \begin{bmatrix}
 u_1 \\
 u_2 \\
 u_3 \\
 u_4 \\
 u_5 \\
 u_6 \\
 u_7 \\
 \vdots \\
 \vdots \\
 u_n
 \end{bmatrix}$$

- For each linear hat test/weighting function  $\phi_i$ , the weak form expression is:

$$u_{j-1} \left( \frac{1}{h_j^2} \right) \cdot h_j + u_j \left( \frac{-1}{h_j^2} \cdot h_j - \frac{1}{h_{j+1}^2} \cdot h_{j+1} \right) + u_{j+1} \left( \frac{1}{h_{j+1}^2} \right) \cdot h_{j+1} = \frac{f_j \cdot h_j}{2} + \frac{f_j \cdot h_{j+1}}{2}$$



# A-Matrix Assembly: Idea Node-by-node assembly

- **Note:** If the grid spacing is uniform, then we recover something that looks a lot like the **finite difference matrix system**:

$$h \cdot \begin{bmatrix} \frac{-1}{h^2} & \frac{1}{h^2} & 0 & 0 & 0 & 0 & \dots & 0 \\ \frac{1}{h^2} & \frac{-2}{h^2} & \frac{1}{h^2} & 0 & 0 & 0 & \dots & 0 \\ 0 & \frac{1}{h^2} & \frac{-2}{h^2} & \frac{1}{h^2} & 0 & 0 & \dots & 0 \\ 0 & 0 & \frac{1}{h^2} & \frac{-2}{h^2} & \frac{1}{h^2} & 0 & \dots & 0 \\ 0 & 0 & 0 & \frac{1}{h^2} & \frac{-2}{h^2} & \frac{1}{h^2} & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ \vdots \\ u_n \end{bmatrix} = h \cdot \begin{bmatrix} f_1/2 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ \vdots \\ f_N/2 \end{bmatrix}$$

- What we need to **also** notice though, is when the grid spacing is not uniform, the differential equation/matrix equation does not require (explicit) mapping!!!

# Boundary Conditions?

- How do we handle the boundary conditions?
  - ① **Dirichlet**: For Dirichlet boundary conditions, we will have to change the appropriate equation in the row of the A-matrix.
  - ② **Neumann**: For Neumann boundary conditions, the natural boundary condition term in the weak form can be used:

$$\underbrace{\phi_i \frac{\partial u}{\partial x} \Big|_{x_L}^{x_R}}_{BCs: [A][u]} = g(x)$$

# How do we code this up: MATLAB Node-by-node assembly?

- The weak form:

$$\underbrace{\phi_i \frac{\partial u}{\partial x} \Big|_{x_L}^{x_R}}_{BCs: [A][u]} - \underbrace{\sum_{j=1}^{NV} u_j \int_{x_L}^{x_R} \left( \frac{\partial \phi_i}{\partial x} \cdot \frac{\partial \phi_j}{\partial x} \right) dx}_{\text{row}_i \text{ of } [A][u] : \text{internal nodes}} = \underbrace{\int_{x_L}^{x_R} (\phi_i f) dx}_{RHS_i}$$

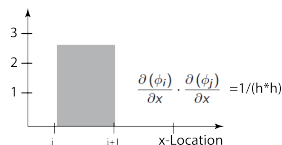
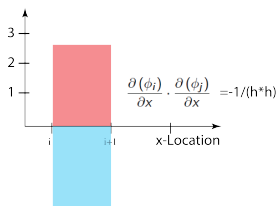
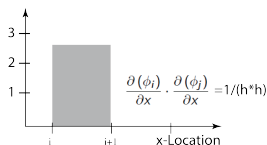
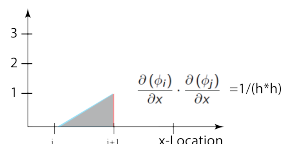
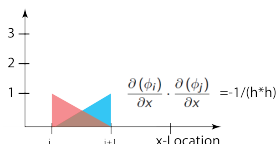
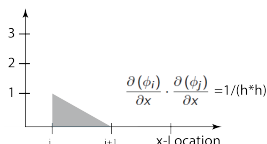
- With a Dirichlet BC, is numerically approximated as:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & \dots \\ \frac{1}{h_1} & \left( \frac{-1}{h_1} + \frac{-1}{h_2} \right) & \frac{1}{h_2} & 0 & 0 & 0 & \dots \\ 0 & \frac{1}{h_2} & \left( \frac{-1}{h_2} + \frac{-1}{h_3} \right) & \frac{1}{h_3} & 0 & 0 & \dots \\ 0 & 0 & \frac{1}{h_3} & \left( \frac{-1}{h_3} + \frac{-1}{h_4} \right) & \frac{1}{h_4} & 0 & \dots \\ 0 & 0 & 0 & \frac{1}{h_4} & \left( \frac{-1}{h_4} + \frac{-1}{h_5} \right) & \frac{1}{h_5} & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \dots \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \\ \vdots \\ u_n \end{bmatrix} = \begin{bmatrix} 0 \\ \frac{f \cdot h_1}{2} + \frac{f \cdot h_2}{2} \\ \frac{f \cdot h_2}{2} + \frac{f \cdot h_3}{2} \\ \frac{f \cdot h_3}{2} + \frac{f \cdot h_4}{2} \\ \frac{f \cdot h_4}{2} + \frac{f \cdot h_5}{2} \\ \frac{f \cdot h_5}{2} + \frac{f \cdot h_6}{2} \\ \frac{f \cdot h_6}{2} + \frac{f \cdot h_7}{2} \\ \vdots \\ 0 \end{bmatrix}$$

- Notice that there is a “block” pattern for each element,  $1/h_j$ !!!

# A-Matrix : Let's Now to Elementwise Construction

- The pattern in the matrix corresponds to the ELEMENTS.
- For each element there is half of a basis function contribution from  $node(j)$  and half a basis function contribution from  $node(j + 1)$



# A-Matrix : Element-wise matrix assembly

- When we look at the (partial) basis function interactions on each element, we get a  $2 \times 2$  element matrix.
- For each element,  $k$  in the 1D domain, we get:

$$ElemMat = -1 * \begin{pmatrix} \frac{\partial \phi_i}{\partial x} \frac{\partial \phi_j}{\partial x} \cdot h_k & \frac{\partial \phi_i}{\partial x} \frac{\partial \phi_{j+1}}{\partial x} \cdot h_k \\ \frac{\partial \phi_{i+1}}{\partial x} \frac{\partial \phi_j}{\partial x} \cdot h_k & \frac{\partial \phi_{i+1}}{\partial x} \frac{\partial \phi_{j+1}}{\partial x} \cdot h_k \end{pmatrix} = \begin{pmatrix} \frac{-1}{h_k} & \frac{1}{h_k} \\ \frac{1}{h_k} & \frac{-1}{h_k} \end{pmatrix} \quad (5)$$

- So, for each element we get entries in the:
  - $A(k+0, k+0) = A(k+0, k+0) - 1/h_k$
  - $A(k+0, k+1) = A(k+0, k+1) + 1/h_k$
  - $A(k+1, k+0) = A(k+1, k+0) + 1/h_k$
  - $A(k+1, k+1) = A(k+1, k+1) - 1/h_k$

locations in the global A-matrix.

# A-Matrix and RHS

- Evaluating the RHS entry for an element,  $k$  gives:

$$ELEM RHS = h_k \cdot \begin{pmatrix} \frac{f_k}{2} \\ \frac{f_k}{2} \end{pmatrix} \quad (6)$$

- This is pretty straightforward. The entries are stamped/located in the global RHS vector in the  $k$  and  $k + 1$  rows.
  - $RHS(k + 0) = RHS(k + 0) + h_k \cdot f_k/2$
  - $RHS(k + 1) = RHS(k + 1) + h_k \cdot f_k/2$

# A-Matrix and RHS

- **Algorithm:** For each element,  $k$ , in the domain, calculate the  $ElemMat$  and  $ELEMRHS$ .
- Stamp the  $ElemMat$  and  $ELEMRHS$  entries into the appropriate locations of  $A$ -matrix and  $RHS$ -vector.

① Add the  $ElemMat$  values to existing global  $A$ -matrix:

$$A(k+0, k+0) = A(k+0, k+0) + ElemMat(1, 1);$$

$$A(k+0, k+1) = A(k+0, k+1) + ElemMat(1, 2);$$

$$A(k+1, k+0) = A(k+1, k+0) + ElemMat(2, 1);$$

$$A(k+1, k+1) = A(k+1, k+1) + ElemMat(2, 2);$$

② Similarly, for the  $RHS$ :

$$RHS(k+0) = RHS(k+0) + ELEMRHS(1)$$

$$RHS(k+1) = RHS(k+1) + ELEMRHS(2)$$

- Finish with the BC's : see next slide.

# Boundary conditions

- Let's now look at boundary conditions:

$$\underbrace{w \frac{\partial u}{\partial x} \Big|_{x_L}^{x_R}}_{BCs} \quad (7)$$

- If  $\frac{\partial u}{\partial x} = 0$  on the boundary, we can assume the above term equals zero.
- If  $u = 0$  on the boundary, we must alter the A-matrix to include this value.



# Solve for the Unknown

- At this point we solve for the unknown,  $u$  and plot the result.

$$[A][u] = [RHS] \rightarrow \boxed{[u] = [A]^{-1}[RHS]} \quad (8)$$

# What have we learned?

- We saw the node-wise and elementwise ways to assemble the A-matrix.
- We saw how to create element-matrices and stamp them into the global matrix.
- FEM has a similar matrix system as FDM/FVM when grid spacing is uniform.
- FEM allows for general node spacing without mappings (Hell yeah!)
- We'll see 2D FEM next time.